Lab – Automatically Remove Docker Containers

To Begin with the Lab

Summary

In this lab, we learn how to automatically remove Docker containers after they have stopped or exited. Normally, when we stop a Docker container, it remains available in Docker even though it is no longer running. Docker provides the --rm flag to automatically remove a container when it exits. This helps keep the Docker environment clean by preventing unnecessary stopped containers from accumulating.

Understanding Docker Container Auto-Removal

When we create and run a Docker container, it continues to exist even after it has stopped. For example, if we stop a container using the docker stop command, Docker does not automatically delete it.

To automatically remove a container after it exits, we can use the --rm flag with the docker run command.

Why do we use the --rm flag?

- **Automatic Removal:** Removes the container automatically after it exits.
- **Clean Environment:** Prevents unnecessary stopped containers from accumulating.
- **Easy Management:** Reduces the need to manually remove containers.
- **Resource Management:** Keeps the list of existing containers organized.

Example:

Suppose we have a Docker image named static-app.

We want to create a container named static-app-instance and automatically remove it when it stops.

Command:

docker run -d --rm --name static-app-instance static-app

Here:

- docker run – Creates and starts a container.
- -d – Runs the container in detached mode.
- --rm – Automatically removes the container after it exits.
- --name – Assigns a custom name to the container.
- static-app-instance – Custom container name.

- static-app – Docker image name.

Result: When the container stops or exits, Docker automatically removes it.

Lab Steps

Step 1: Check Existing Containers

First, display all existing containers.

```
docker ps -a
```



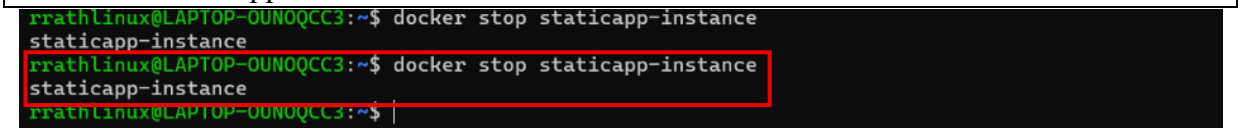
Explanation:

- Displays running and stopped containers.
- Identify the existing static-app-instance container.

Step 2: Remove the Existing Container

If the previous container still exists, remove it before creating another container with the same name.

```
docker rm static-app-instance
```



Explanation:

- docker rm – Removes a stopped container.
- static-app-instance – Specifies the container name.

Note: If the container is running, stop it first:

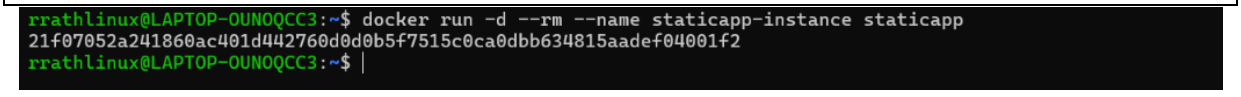
docker stop static-app-instance

Then remove it.

Step 3: Create a Container Using the --rm Flag

Now create a new container with automatic removal enabled.

```
docker run -d --rm --name static-app-instance static-app
```



Explanation:

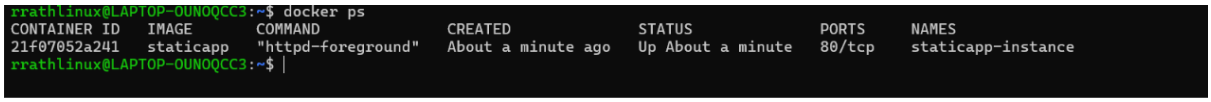
- Creates a new container from the static-app image.
- Assigns the name static-app-instance.
- Runs the container in detached mode.
- Automatically removes the container when it exits.

Step 4: Verify the Running Container

Execute:

```
docker ps
```

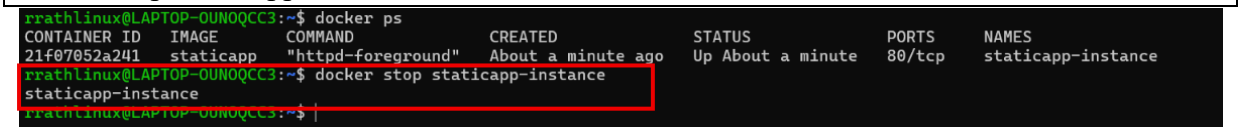
You should see the newly created container in the running state.



Step 5: Stop the Container

Now stop the container using its custom name.

```
docker stop static-app-instance
```



Explanation:

- Stops the running container.
- Because we used the --rm flag, Docker automatically removes the container after it stops.

Step 6: Verify Automatic Removal

Execute:

```
docker ps -a
```



Expected Result:

The static-app-instance container will no longer appear in the container list because Docker has automatically removed it.